

Assignment-1

Object-Oriented Programming & Inheritance Concepts in Java

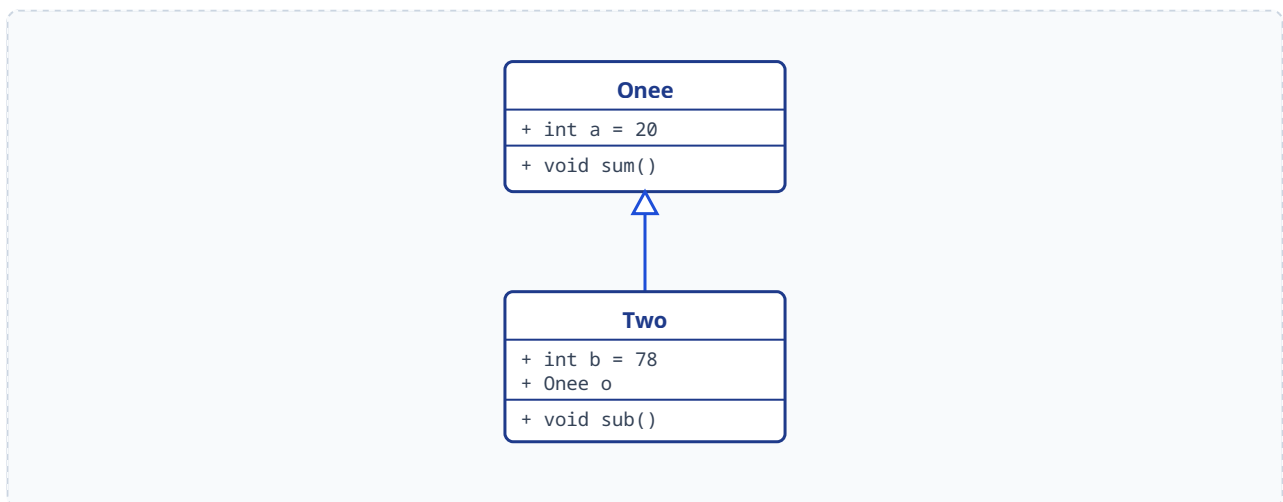
Subject: Java Programming Lab | **Topic:** Inheritance Types & Interface Implementation

1. Single Inheritance Concept Single Inheritance

1. STEP-BY-STEP PROCESS & EXECUTION FLOW

1. **Class Definition:** Define class Onee containing instance variable `a = 20` and method `sum()`.
2. **Inheritance:** Define class Two that extends Onee using the `extends` keyword. Class Two inherits member `a` and method `sum()`, while defining its own variable `b = 78` and method `sub()`.
3. **Object Instantiation:** Inside `Single.main()`, create an object `t` of child class Two.
4. **Access Members:** Access inherited field `t.a` and own field `t.b`, printing their values.
5. **Method Invocation:** Call inherited method `t.sum()` (calculates $20 + 30 = 50$) and child method `t.sub()` (calculates $78 - 9 = 69$).

2. UML CLASS DIAGRAM



3. CODE EXPLANATION

This program demonstrates **Single Inheritance** in Java, where class `Two` derives directly from base class `Onee`. Through inheritance, object `t` of type `Two` acquires direct access to member variables and methods of class `Onee` without explicit redefinition.

4. SOURCE CODE

```
package lab;

class Onee {
    int a = 20;
    void sum() {
        System.out.println("sum is:" + (a+30));
    }
}

class Two extends Onee {
    int b = 78;
    Onee o = new Onee();
    void sub() {
        System.out.println("sub is:" + (b-9));
    }
}
```

```
    }  
}  
  
public class Single {  
    public static void main(String[] args) {  
        Two t = new Two();  
        System.out.println("can access:" + t.a);  
        System.out.println("can:" + t.b);  
        t.sum();  
        t.sub();  
    }  
}
```

5. EXPECTED OUTPUT

```
can access:20  
can:78  
sum is:50  
sub is:69
```

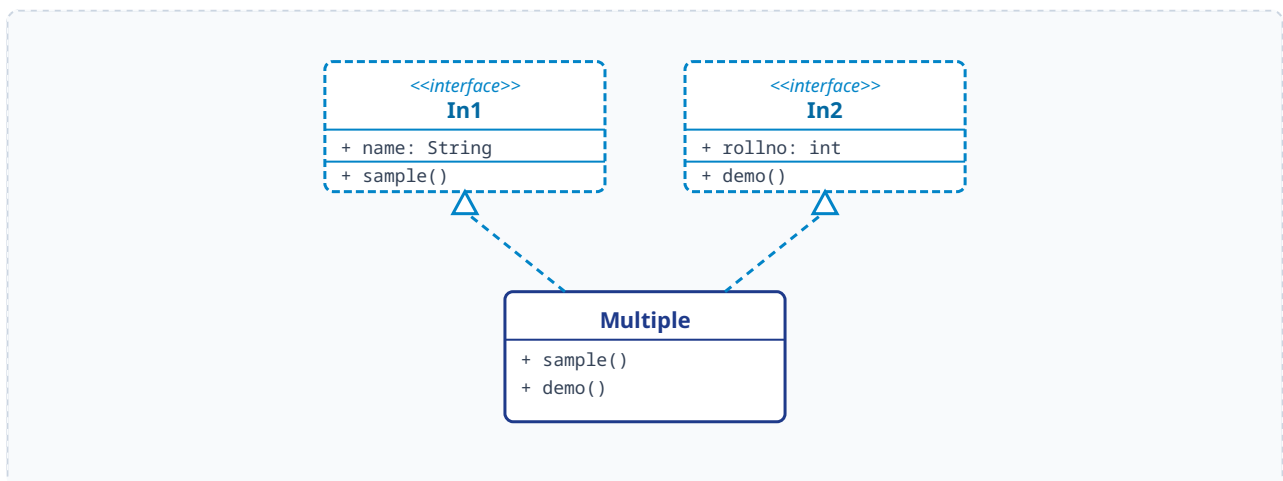
2. Multiple Inheritance via Interfaces

Multiple Inheritance

1. STEP-BY-STEP PROCESS & EXECUTION FLOW

1. **Interface Declarations:** Define interface In1 with variable name and abstract method sample(), and interface In2 with variable rollno and abstract method demo().
2. **Multiple Interface Implementation:** Class Multiple implements both In1 and In2 using comma separation (implements In1, In2).
3. **Method Overriding:** Class Multiple provides concrete implementations for sample() and demo(), marking them with public access modifier.
4. **Execution:** In main(), instantiate Multiple and call both methods to print student profile information.

2. UML CLASS DIAGRAM



3. CODE EXPLANATION

Java does not support multiple inheritance with classes to avoid ambiguity (the diamond problem). However, multiple inheritance is achieved using **Interfaces**. The class Multiple fulfills contracts set by both In1 and In2 interfaces by implementing their respective abstract methods.

4. SOURCE CODE

```
package lab;

interface In1 {
    String name = "Revanth";
    void sample();
}

interface In2 {
    int rollno = 221;
    void demo();
}

public class Multiple implements In1, In2 {
```

```
public void sample() {  
    System.out.println(name + "\nHe is second year");  
}  
public void demo() {  
    System.out.println("From CSM-D");  
}  
public static void main(String[] args) {  
    Multiple m = new Multiple();  
    m.sample();  
    m.demo();  
}  
}
```

5. EXPECTED OUTPUT

```
Revanth  
He is second year  
From CSM-D
```

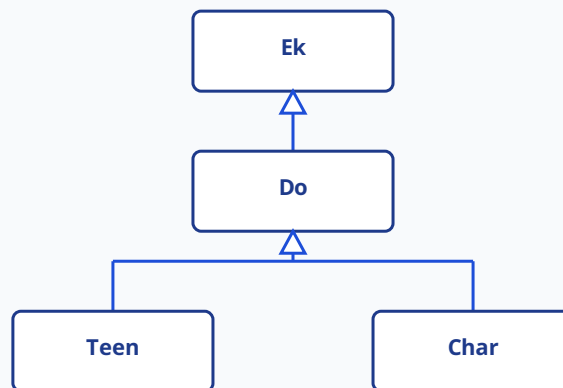
3. Hybrid Inheritance Pattern

Multilevel + Hierarchical

1. STEP-BY-STEP PROCESS & EXECUTION FLOW

1. **Base & Multilevel Chain:** Define class Ek. Class Do extends Ek. Class Teen extends Do (forming Multilevel inheritance).
2. **Hierarchical Branch:** Class Char also extends Do (forming Hierarchical inheritance from parent Do).
3. **Object Execution:** In main(), create instances of Char, Teen, and Do.
4. **Method Calls:** Object d calls innocent() (inherited from Ek) and playboy(). Object t calls chickmagnet(). Object c calls fire().

2. UML CLASS DIAGRAM



3. CODE EXPLANATION

Hybrid Inheritance is a combination of two or more inheritance types. In this design, Multilevel inheritance (Ek → Do → Teen) combines with Hierarchical inheritance (Do → Teen and Do → Char).

4. SOURCE CODE

```
package lab;

class Ek {
    String name = "Revanth";
    void innocent() {
        System.out.println(name + "\n HE IS INNOCENT");
    }
}

class Do extends Ek {
    String name = "Venu";
    void playboy() {
        System.out.println(name + "\n HE IS PLAYBOY");
        System.out.println("INNOCENT AND PLAYBOY ARE DARLINGS ");
    }
}
```

```

    }
}

class Teen extends Do {
    String name = "Jathin";
    void chickmagnet() {
        System.out.println(name + "\n GIRLS FAVOURITE");
        System.out.println("PLAYBOY AND CHICKMAGNET ARE SALEY'S");
    }
}

class Char extends Do {
    String name = "Srikar";
    void fire() {
        System.out.println(name + "\n SHORT TEMPER ");
        System.out.println("PLAYBOY AND SHORT TEMPER ARE BEST FRIENDS");
    }
}

public class Hybrid {
    public static void main(String[] args) {
        Char c = new Char();
        Teen t = new Teen();
        Do d = new Do();
        d.innocent();
        d.playboy();
        t.chickmagnet();
        c.fire();
    }
}

```

5. EXPECTED OUTPUT

```

Revanth
  HE IS INNOCENT
Venu
  HE IS PLAYBOY
INNOCENT AND PLAYBOY ARE DARLINGS
Jathin
  GIRLS FAVOURITE
PLAYBOY AND CHICKMAGNET ARE SALEY'S
Srikar
  SHORT TEMPER
PLAYBOY AND SHORT TEMPER ARE BEST FRIENDS

```

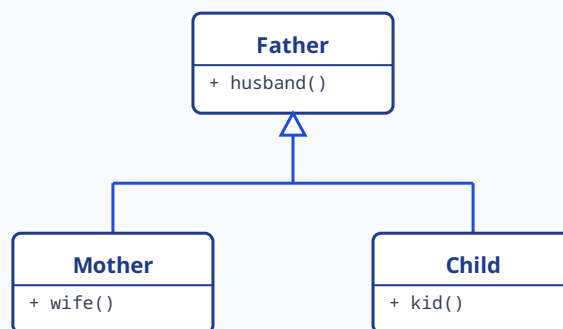
4. Hierarchical Inheritance Structure

Hierarchical Inheritance

1. STEP-BY-STEP PROCESS & EXECUTION FLOW

1. **Base Class Definition:** Define class `Father` with attributes `name` and `age`, and method `husband()`.
2. **Multiple Derived Classes:** Derive class `Mother` from `Father`, overriding local attributes and defining method `wife()`. Derive class `Child` from `Father`, defining method `kid()`.
3. **Instantiation & Execution:** Create objects `c` of `Child` and `m` of `Mother`.
4. **Method Calls:** Call `c.husband()` (inherited from `Father`), `m.wife()`, and `c.kid()` to execute statements.

2. UML CLASS DIAGRAM



3. CODE EXPLANATION

In **Hierarchical Inheritance**, multiple subclass branches (`Mother` and `Child`) inherit directly from a single parent superclass (`Father`). Both subclasses access parent members while adding specialized features.

4. SOURCE CODE

```
package lab;

class Father {
    String name = "GURESWARA REDDY\n";
    int age = 43;
    void husband() {
        System.out.println(name + age + "\nI AM HEAD OF THE HOUSE\n");
    }
}

class Mother extends Father {
    String name = "PAVANI\n";
    int age = 36;
    void wife() {
        System.out.println(name + age + "\nI AM THE HOMEMINISTER OF THE HOUSE\n");
    }
}
```



```

    }
}

class Child extends Father {
    String name = "REVANTH REDDY\n";
    int age = 18;
    void kid() {
        System.out.println(name + age + "\nI AM A SINGLE KID\n");
    }
}

public class Hiarcheairical {
    public static void main(String[] args) {
        Child c = new Child();
        Mother m = new Mother();
        c.husband();
        m.wife();
        c.kid();
    }
}

```

5. EXPECTED OUTPUT

```

GURESWARA REDDY
43
I AM HEAD OF THE HOUSE

PAVANI
36
I AM THE HOMEMINISTER OF THE HOUSE

REVANTH REDDY
18
I AM A SINGLE KID

```